MOVIE RECOMMENDATION SYSTEM

Machine Learning Project

- **Problem Statement –** Given one movie a user liked, recommend the 5 most similar movies
- **Theme –** Recommender Systems (Content-Based Filtering)
- **Category –** Software
- **Guided by –** Ansh Agrawal
- **Team Members –**
  - Shakti Subham Mishra (1ETBTA20251040)
  - Apurba Mohanty (1ETBTA20251039)
  - Preksha Singh (1ETBTA20251038)
  - Puravi Dash (1ETBTA20251041)



Sri Sri University

CONTENT-BASED MOVIE RECOMMENDER

❖ Proposed Solution: recommend similar movies from their content

Detailed explanation of the proposed solution

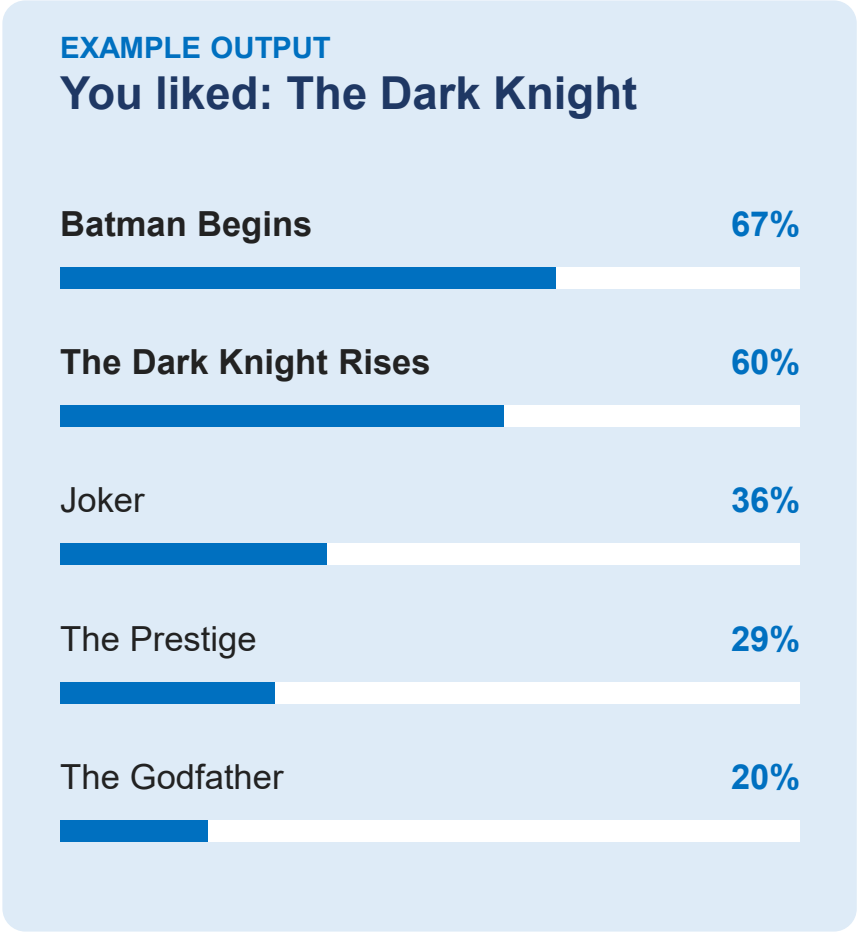
- **Tags:** each movie is described by its genres + director + 2 lead actors + 4–6 plot keywords
- **Vectors:** tags become word-count vectors (bag of words, 537-word vocabulary)
- **Similarity:** cosine similarity between every pair of movies (110 × 110 table)
- **Output:** user types a movie, the system returns the top 5 with a match %

How it addresses the problem

- Ends choice overload: one title in, five relevant suggestions out
- Needs no ratings or watch history, so it works from day one, even for new movies (no cold start)

Innovation and uniqueness of the solution

- People become single tokens (christophernolan), so names never get mixed up
- Typo-tolerant search: "toy stroy" still finds Toy Story
- Explainable: every suggestion can be justified by the words two movies share



TECHNICAL APPROACH

Technologies to be used

Python 3
programming language

pandas
load & clean the CSV

scikit-learn
CountVectorizer,
cosine_similarity

difflib
typo-tolerant title search

Laptop CPU
no GPU, no cloud needed

Methodology and process for implementation

1. Load data
movies.csv
110 movies × 6 fields

→

2. Build tags
genres + director
+ cast + keywords

→

3. Vectorise
CountVectorizer
537 word counts

→

4. Similarity
cosine similarity
110 × 110 table

→

5. Recommend
sort the row,
return top 5 + match %

How similarity is measured

$$\cos(A, B) = (A \cdot B) / (|A| \times |B|)$$

1 = identical tags, 0 = nothing in common

The Dark Knight (11 words) vs Batman Begins (10 words): 7 shared words

$7 / \sqrt{(11 \times 10)} = 0.67 \rightarrow$ **67% match**

Core code (movie_recommender.py)

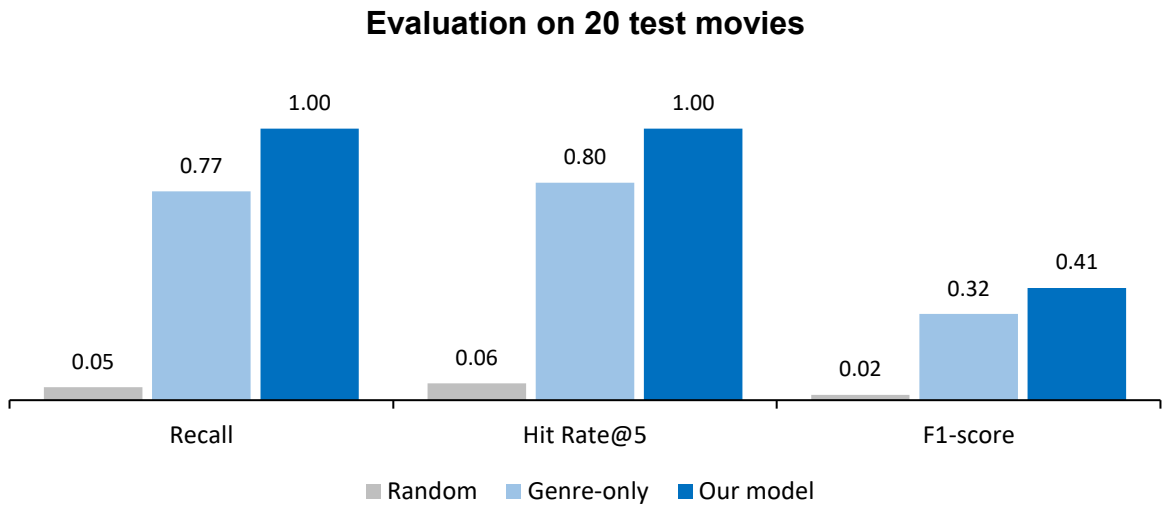
```
vectors = CountVectorizer().fit_transform(df["tags"])
sim = cosine_similarity(vectors)          # 110 x 110

i = df.index[df["title"] == title][0]
ranked = sorted(enumerate(sim[i]), key=lambda p: -p[1])
top5 = [df["title"][j] for j, _ in ranked if j != i][:5]
```

FEASIBILITY AND VIABILITY

Analysis of the feasibility of the idea

- **Technical:** ~100 lines of Python, only free open-source libraries
- **Speed:** full run and evaluation in under 2 s on a laptop CPU; no GPU, no training loop
- **Data:** 110 movies × 6 fields; extend by adding rows to a CSV
- **Proven:** 20 test queries, 26 expected movies: all 26 found (Recall 1.00, F1 0.41)



Potential challenges and risks → Strategies for overcoming them

Challenge / risk	Strategy
Quality depends on hand-written tags	Extract keywords automatically from plot summaries (TF-IDF or sentence embeddings)
Recommends "more of the same"	Mix in diverse or popular picks; add user ratings for a hybrid recommender
Similarity table grows as n^2 for big catalogues	Use a nearest-neighbour index instead of the full table
Small, self-labelled test set	Re-evaluate on public datasets with real ratings (MovieLens, TMDB 5000)

IMPACT AND BENEFITS

Potential impact on the target audience

Movie viewers

Less time scrolling; one liked movie leads to five similar ones to watch next

Small & regional platforms

Recommendations without big user data, GPUs or paid services; film clubs and libraries too

Students & educators

A clear, explainable first ML recommender that runs on any laptop

Benefits of the solution

1. **Social**
   Helps viewers discover Indian and world cinema alongside Hollywood; reduces choice fatigue

2. **Economic**
   Free open-source stack on existing hardware; better suggestions can keep viewers engaged

3. **Environmental**
   CPU only with no model training, so energy use is tiny compared with GPU-trained recommenders

4. **Trust & privacy**
   Every suggestion is explainable, and no personal data or watch history is collected

RESEARCH AND REFERENCES

Research work done

- **Compared** popularity, collaborative and content-based approaches; chose content-based (no ratings needed)
- **Built** our own dataset: 110 movies, 20 genres, 64 directors
- **Labelled** 20 test movies with 26 expected recommendations
- **Benchmarked** against random and genre-only baselines
- **Result:** Recall 1.00 vs 0.77 (genre-only) and 0.05 (random)

Details / links of the references

[1] F. Ricci, L. Rokach, B. Shapira, Recommender Systems Handbook, Springer, 2011

[2] M. J. Pazzani, D. Billsus, "Content-Based Recommendation Systems", The Adaptive Web, Springer, 2007

[3] C. D. Manning, P. Raghavan, H. Schütze, Introduction to Information Retrieval, Cambridge Univ. Press, 2008

[4] Y. Koren, R. Bell, C. Volinsky, "Matrix Factorization Techniques for Recommender Systems", IEEE Computer, 2009

[5] X. He et al., "Neural Collaborative Filtering", Proc. WWW, 2017

[6] F. M. Harper, J. A. Konstan, "The MovieLens Datasets: History and Context", ACM TiiS, 2015

[7] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", JMLR, 2011

LINKS

scikit-learn documentation: https://scikit-learn.org

MovieLens datasets: https://grouplens.org/datasets/movielens/

Project code: movie_recommender.py + movies.csv (submitted with this deck)